Bridge-First Verification and Mechanistic Evidence Chains

A conceptual primer for future work on LeanPeTTa, the MeTTa suite, and chain-based evidence reasoning

2026-03-19

This note distills the most useful technical ideas developed in the associated design and proof conversations. It is intentionally conceptual rather than historical. Its purpose is to preserve the architecture, proof style, theorem surfaces, and modeling ideas that proved most valuable, so future chats can resume from a clean, explicit foundation rather than from scattered local context.

Scope. The note covers: (1) the layered architecture for the MeTTa suite and LeanPeTTa, (2) a bridge-first proof methodology for executable runtimes and theorem-bearing references, (3) the deterministic-to-reference proof problem and its clean decomposition, and (4) a chain-first view of mechanistic evidence reasoning, especially for biological gene-prioritization settings.

Contents

1 Executive summary **3**

2 The MeTTa-suite architecture **3**
2.1 The layered repository shape . . . 3
2.2 What belongs where . . . 4

3 The LeanPeTTa architecture inside Algorithms **4**
3.1 The four-path split . . . 4
3.2 The key theorem-bearing modules . . . 4

4 Bridge-first proof methodology **5**
4.1 Why proving against the live runtime is often the wrong target . . . 5
4.2 Supported-first theorem surfaces . . . 5
4.3 Split invariants by phase . . . 6

5 Canonical semantics and the role of ReferenceEval **6**
5.1 Thin concrete wrappers beat copied proofs . . . 7

6 The compositional match adequacy story **7**
6.1 The right target . . . 7
6.2 The missing fold invariant . . . 7

7 Deterministic versus reference evaluation **7**
7.1 The real theorem surface . . . 7
7.2 Pre-args and post-args dispatch phases . . . 8
7.3 Exact-control lemmas belong at the definition site . . . 8

8 Depth-indexed simulation and safe traces **8**
8.1 Why depth matters . . . 9
8.2 Safe traces . . . 9

9 Runtime booleans versus theorem-facing semantic predicates **9**

10 State agreement and memo invariants **10**
10.1 Constant-state fragments . . . 10
10.2 Memo as a loop invariant . . . 10

11 Representation choices that make theorems trivial **10**

12 Tests, stubs, and proof timing **10**

13 Chain-first reasoning for mechanistic evidence **11**
13.1 From flat features to typed paths . . . 11
13.2 Rescue cases as mechanism discovery . . . 11
13.3 Chain-first scoring for PLN-style systems . . . 11

14 A practical primer for future chats **12**
14.1 Architecture checklist 12
14.2 Proof-design checklist 12
14.3 When to refactor 12

15 Recommended near-term roadmap **12**

16 Conclusion **13**

1. Executive summary

The central lesson is simple: when an executable runtime is operationally useful but proof-hostile, the right response is usually *not* to prove directly against the opaque implementation. The right response is to introduce a *semantic waist*: a small theorem-facing interface where executable behavior, total reference behavior, and higher-level specifications can meet cleanly.

For LeanPeTTa and the surrounding MeTTa suite, this leads to five durable principles.

1. **Prove at the bridge, not at the opaque implementation.** The executable runtime may remain optimized, partial, or operationally structured. Proof ownership should move to a transparent bridge layer.
2. **Use supported-first theorem surfaces.** Instead of demanding one global equivalence theorem too early, state local semantic witnesses for covered fragments and prove unconditional theorems from those witnesses.
3. **Split invariants by phase.** Large proofs become tractable when decomposed into branch classification, local semantic atom agreement, continuation/simulation, and packaging.
4. **Distinguish runtime gates from semantic predicates.** Booleans that are useful for execution are not automatically strong enough as proof assumptions. They often need theorem-facing semantic counterparts.
5. **For evidence modeling, score chains rather than isolated features.** In applications such as gene prioritization, the scientific object of interest is often a typed evidence path, not a flat score.

This note turns those principles into concrete guidance.

2. The MeTTa-suite architecture

2.1. The layered repository shape

The clean suite-level architecture is:

1. **MeTTailCore**: a small, neutral base layer containing syntax, matching, substitution, lookup-plan vocabulary, and other genuinely shared substrate-level notions.
2. **Algorithms**: executable Lean runtimes, lowerings, theorem-bearing total and faithful references, and local refinement/adequacy machinery that does not depend on richer external semantics.
3. **Mettapedia**: the semantic authority for derived specifications, language-level contract catalogs, conformance theorems, and richer declarative models.
4. **Backend consumers**: downstream artifact consumers such as Rust executors or other runtime systems, which should consume exported artifacts rather than define competing semantics.

The crucial import law is:

- MeTTailCore imports neither Algorithms nor Mettapedia.
- Algorithms may import MeTTailCore, but never Mettapedia.

- Mettapedia may import Algorithms and MeTTailCore.
- Only conformance or bridge layers in Mettapedia should import executable runtime layers from Algorithms.

This keeps executable code below, semantic authority above, and neutral substrate logic in the middle.

2.2. What belongs where

A recurring source of confusion is the placement of "execution contracts" or "outsourcing contracts." The clean rule is:

- **Neutral schema** belongs in the shared substrate layer only if it is truly language-agnostic.
- **Language-specific semantic catalogs** belong in Mettapedia, because they are proved facts about the language, not merely runtime settings.
- **Executable runtime exposure** belongs in Algorithms.
- **Soundness or conformance of runtime contracts** belongs in Mettapedia.

A practical check is: if a concept is part of the proved semantic story of a language, it probably should not be pushed downward merely because it is useful to an optimizer.

3. The LeanPeTTa architecture inside Algorithms

3.1. The four-path split

The LeanPeTTa runtime/proof stack is best understood as four interacting paths:

1. **Live executable path**: the real runtime API and operational evaluator.
2. **Total fuel-indexed reference path**: a theorem-bearing total kernel with explicit fuel.
3. **Faithful status path**: a reference that separates semantic results from out-of-fuel or failure-style states.
4. **Adequacy and refinement layers**: theorem packaging that connects the live/runtime path to the theorem-bearing references.

The big architectural correction was to stop treating the old live partial-def strongly connected component as the proof center. The executable path remains useful and important, but theorem ownership belongs at the total and faithful waist.

3.2. The key theorem-bearing modules

A clean conceptual split is:

- `Session.lean`: executable definitions, public wrappers, and only small local semantic lemmas.
- `SessionReferenceTotal.lean`: public total reference backend and unconditional preservation theorems.

- `SessionReferenceFaithful.lean`: status-based faithful backend and agreement with the total backend on successful runs.
- `SessionReferenceAdequacy.lean`: packaging of local adequacy witnesses into public theorems.
- `SessionRefinement.lean` and `SessionRefinementTotal.lean`: transport theorems from live/runtime behavior to reference behavior and then to the total backend.

The top theorem files should remain boring. If they become the proof battlefield, the architecture has drifted.

4. Bridge-first proof methodology

4.1. Why proving against the live runtime is often the wrong target

Executable evaluators often have the wrong proof shape. They may be:

- mutually recursive,
- partial or operationally optimized,
- memoized,
- deeply branchy,
- or structured around work queues rather than semantically natural recursion.

In such situations, direct induction or direct unfolding usually creates high-cost proof obligations with little reuse. A better approach is to identify a *bridge waist*: a small set of transparent semantic interfaces that represent what the runtime is doing at a proof-friendly level.

For LeanPeTTa, the useful bridge surfaces included:

- `referenceMatchBindings`,
- `evalMatchedTemplate`,
- `referenceMatchIntrinsicResult`,
- `intrinsicMatchResultWithEval`,
- the total and faithful reference evaluators,
- and a canonical abstract reference evaluator in `ReferenceEval.lean`.

4.2. Supported-first theorem surfaces

A recurring failure mode is asking for a theorem that is semantically too strong too early. The antidote is a *supported-first* theorem surface.

Instead of trying to prove a single theorem of the form

> "the optimized runtime equals the full reference runtime on all accepted terms,"

introduce a witness that says:

> "this term or fragment is supported by the bridge assumptions needed for the proof."

Then prove an unconditional theorem from that witness. If needed, separately prove that the runtime gate implies the supported witness, or strengthen the runtime gate if it does not.

This avoids wasting time on a false or over-strong theorem surface.

4.3. Split invariants by phase

The most useful invariant split for LeanPeTTa was:

1. **Binding enumeration equality**: the two sides enumerate the same candidate bindings.
2. **Substituted-template evaluation equality**: the two sides evaluate bound templates the same way on a controlled fragment.
3. **Intrinsic equality**: local semantic equality at the match or intrinsic waist.
4. **Whole-evaluator transport**: package the local equality into a public refinement theorem.

A similar phase split reappears in the deterministic bridge problem:

1. classify the dispatch branch,
2. prove the local semantic atom agrees,
3. prove the continuation or simulation step,
4. package the result into the top-level refinement theorem.

Monolithic invariants tend to hide the real source of failure. Split invariants reveal it.

5. Canonical semantics and the role of ReferenceEval

One of the most important cleanups was recognizing that the abstract evaluator in `ReferenceEval.lean` should be the *canonical* proof-facing semantics.

The key lesson is that copied concrete helpers may look convenient, but if the real reference evaluator already factors through an abstract interface, then the proof should target that interface rather than proving equivalence between multiple copies of the same operational idea.

For LeanPeTTa, the right move was:

1. define a public named interface such as `referenceEvalInterfaceN fuel`,
2. define `evalWithStateCoreN` through `ReferenceEval.evalWithStateCore` instantiated at that interface,
3. state bridge predicates in terms of `ReferenceEval.runNestedEffects (referenceEvalInterfaceN fuel)` and `(referenceEvalInterfaceN fuel).intrinsicStateful`,
4. and instantiate the abstract one-step theorems rather than re-proving them concretely.

This was a decisive reduction in proof complexity.

5.1. Thin concrete wrappers beat copied proofs

Once the canonical interface exists, concrete theorems should be wrappers of the form:

> "`simpa [evalWithStateCoreN extunderscore succ] using ReferenceEval.some extunderscore theorem (I := referenceEvalInterfaceN fuel) ...`"

This is the right level of proof reuse. It keeps the evaluator theory centralized and prevents duplicated machine proofs from drifting apart.

6. The compositional match adequacy story

6.1. The right target

A large amount of proof effort initially targeted "match adequacy" too directly. The cleaner shape turned out to be:

- prove binding-enumeration equality first,
- then prove substituted-template evaluator equality for a tiny family,
- then let a generic compositional theorem in the space semantics lift that to intrinsic match equality,
- then package that as a local refinement theorem.

The first successful nontrivial family was a get-atoms-style template family on a small max-nodes fragment. The important insight was that get-atoms was not intrinsically important; it was just the cheapest first family whose semantics made the fold invariant obvious.

6.2. The missing fold invariant

The difficult part of the first match theorem was not the abstract match boundary itself. It was the fold over bindings, which required a local theorem strong enough to show that evaluating the substituted template left the session unchanged on the relevant fragment.

That led to the right local target:

> prove the smallest state-self theorem in the exact shape the fold needs, then reuse the generic compositional theorem.

This pattern is reusable: if a theorem fails deep inside a fold, look for the missing local state or shape invariant before trying a bigger induction.

7. Deterministic versus reference evaluation

7.1. The real theorem surface

The later proof bottleneck was a theorem informally of the form:

> "on deterministic-accepted terms, the optimized evaluator agrees with the reference evaluator."

The key conceptual correction was that this theorem should not be attacked directly. It naturally splits into two problems:

1. **Soundness**: the result chosen by the deterministic evaluator is a result produced by the reference evaluator.
2. **Singleton or uniqueness**: on the supported fragment, the reference evaluator produces exactly one result, namely that same result.

This split is crucial. If a proof fails, one immediately learns whether the deterministic branch is unreachable on the reference side, or whether the reference side still has extra results.

7.2. Pre-args and post-args dispatch phases

A second major correction was recognizing that the branch classifier must mirror the actual operational phases.

For apply terms, there is typically a distinction between:

- **pre-args dispatch** on the raw term, such as translation checks or memo decisions, and
- **post-args dispatch** on the term after argument evaluation, such as builtin dispatch, first-rule reduction, partial-arity cases, or unchanged cases.

Classifying all branches in one phase loses that distinction and makes proofs harder than necessary.

7.3. Exact-control lemmas belong at the definition site

One of the most reusable insights was that exact-control theorems should be proved in the file where the relevant private definition lives.

For example:

- exact branch-decomposition theorems for a deterministic evaluator belong in its defining file,
- exact one-step queue theorems for the reference evaluator belong in `ReferenceEval.lean`,
- the bridge file should consume those theorems, not reopen the definitions.

This prevents the bridge from turning into a second copy of the evaluator.

8. Depth-indexed simulation and safe traces

A central obstacle in deterministic-versus-reference proofs is that the two evaluators may recurse in different ways:

- one may use recursive descent over terms and arguments,
- the other may use an explicit work queue with depth indices.

The most successful conceptual response is to define a *depth-indexed reference entrypoint* that starts the reference evaluator at an arbitrary queue depth. Then one proves one-step theorems at that level and inducts over a supported trace witness instead of raw fuel.

8.1. Why depth matters

Once one-step theorems are stated at "current depth," the reference side after a single step is again of the same form, just at depth $d + 1$. That aligns the operational shape of the reference evaluator with the recursive shape of the deterministic evaluator.

8.2. Safe traces

A particularly clean induction surface is an inductive *safe trace* relation describing the currently covered fragment. For example, one may have constructors for:

- unchanged one-step outcomes,
- direct intrinsic one-step outcomes,
- partial builtin or partial-arity outcomes,
- and a recursive constructor extending a trace by one more direct step.

Then simulation is proved by induction on the trace witness, with depth and remaining deterministic fuel as parameters. This is more informative and more modular than induction on a fuel counter alone.

9. Runtime booleans versus theorem-facing semantic predicates

Execution gates often expose booleans such as:

- no blocking rewrites,
- no core builtin overrides,
- unique root choices,
- resolved deterministic results,
- accepted unchanged deterministic terms.

These are useful operationally, but they are not automatically theorem-strength assumptions. A boolean like "no core builtin overrides" may be only a runtime flag unless it is connected to a semantic proposition that actually states the extensional property one needs.

The right pattern is:

1. keep runtime booleans for execution,
2. define theorem-facing semantic predicates such as "core intrinsic direct is singleton-valued,"
3. use the semantic predicates in bridge theorems,
4. separately prove that a trusted construction or runtime invariant implies the semantic predicates.

This avoids smuggling semantic content through booleans that do not yet justify it.

10. State agreement and memo invariants

10.1. Constant-state fragments

For the initial safe fragment of the deterministic bridge, session state does not change. This is a valuable simplification and should be exploited aggressively.

If the branch assumptions include:

- root nested-effects passthrough,
- no stateful intrinsic firing,
- and pure step semantics,

then the current session can often be frozen to the initial session throughout the trace. One should use a constant-session invariant in this regime instead of prematurely introducing a more general relational state invariant.

10.2. Memo as a loop invariant

Memoization is best handled as an explicit invariant. A useful shape is:

> for every cached pair $(call, out)$, the canonical reference evaluator returns exactly $(s, [out])$ from the base session.

Once memo soundness is formulated this way, cache-hit cases become straightforward: use the invariant for hits, prove insert preservation for misses, and keep the memo story orthogonal to the main semantic branch proof.

11. Representation choices that make theorems trivial

A recurrent proof-engineering lesson was that some theorem bottlenecks should be solved by changing the local representation, not by increasing proof power.

For example, if a large function returns either no rows or a singleton row, then representing it directly as a list-of-lists invites unnecessary length proofs. A cleaner local representation is often an option-valued row builder plus a tiny wrapper back into list form. Then the "length at most one" theorem becomes structural.

This is a general lesson: if a proof is repeatedly expensive, ask whether the local definition is exposing the wrong algebraic shape.

12. Tests, stubs, and proof timing

A useful operational rule emerged:

- passing targeted tests does not prove correctness, but it does make proof work reasonable,
- failing targeted tests are a strong sign not to prove that feature yet,
- implementation completeness and proof completeness should be tracked as separate workstreams.

This suggests a disciplined development order:

1. for a feature with intended runtime behavior, add or fix targeted tests first,
2. implement until those tests are green,
3. only then attempt the corresponding theorem.

Meanwhile, proof-bearing supported fragments can continue to expand independently on already green parts of the runtime.

13. Chain-first reasoning for mechanistic evidence

The later evidence-modeling discussion surfaced a parallel conceptual lesson: in many scientific applications, especially biological gene prioritization, a scalar score is not the most meaningful object. The meaningful object is often a *typed evidence chain*.

13.1. From flat features to typed paths

Instead of reasoning only with a feature vector, one can reason with path families such as:

- variant → molecular QTL → gene → trait,
- variant → enhancer or chromatin contact → promoter → gene,
- variant → coding or pathogenicity effect → gene,
- variant → distance-based fallback → gene.

This makes the reasoning process more legible to biologists and more natural for symbolic or probabilistic reasoning systems.

13.2. Rescue cases as mechanism discovery

When a model outperforms a standard baseline, the scientifically interesting question is often not "how much average AUC improved?" but "which loci were rescued, and by what mechanism?"

That naturally leads to a *rescue atlas*: for each rescued case, reconstruct the strongest evidence chain and cluster rescue cases into mechanism archetypes. This is more compelling than reporting only aggregate metrics.

13.3. Chain-first scoring for PLN-style systems

From a reasoning perspective, the right abstraction is often a weighted path grammar with:

- typed edge families,
- overlap-aware support accumulation,
- contradiction penalties,
- and explicit mechanism archetypes.

This connects symbolic chaining, probabilistic reasoning, and interpretable mechanism modeling.

14. A practical primer for future chats

The following checklist is intended to restart future work quickly.

14.1. Architecture checklist

1. Keep runtime implementation in Algorithms.
2. Keep semantic authority and derived language-level catalogs in Mettapedia.
3. Keep only truly neutral foundations in MeTTailCore.
4. Avoid putting proof load in top-level refinement files.
5. Prefer canonical abstract evaluators to copied concrete semantics.

14.2. Proof-design checklist

1. Ask whether the current theorem surface may be too strong.
2. If yes, define a supported witness first.
3. Split branch classification, local atom agreement, continuation, and packaging.
4. Add exact-control theorems where private definitions live.
5. Use wrapper theorems to instantiate abstract results concretely.
6. Prefer induction on a trace witness or semantic object over raw fuel, when operational recursion schemes differ.
7. Keep state invariants as weak as possible for the current fragment.

14.3. When to refactor

A structural refactor is justified only if it does one of the following:

- exposes a new transparent semantic waist shared by the live and reference paths,
- removes an entire class of downstream hypotheses,
- turns repeated proof fights into reusable local theorems,
- or reveals the correct canonical semantics more clearly.

Otherwise, prefer local theorem work over another architectural change.

15. Recommended near-term roadmap

A clean near-term roadmap for LeanPeTTa-style work is:

1. finish any remaining local bridge lemmas for the currently safe deterministic fragment,
2. add the depth-indexed reference entrypoint and one-step singleton/unchanged theorems,

3. prove a supported deterministic-to-reference theorem by induction on safe traces,
4. collapse parameterized refinement theorems into unconditional public theorems for the covered fragment,
5. extend coverage family by family, keeping the same proof architecture,
6. separately implement and test any still-stubbed runtime features before proving them.

For evidence-chaining work, the parallel roadmap is:

1. treat rescue cases as the central object of explanation,
2. disaggregate pooled evidence into typed path families,
3. move from flat-feature scoring toward path-based support models,
4. and evaluate not only rank performance but mechanistic coherence.

16. Conclusion

The most durable idea in this note is that both verification and scientific evidence analysis benefit from the same discipline: identify the right semantic waist, keep theorem surfaces honest, and reason over structured paths rather than over opaque operational detail.

For LeanPeTTa, that means bridge-first verification, supported-first adequacy, canonical reference semantics, and phase-split simulation. For mechanistic evidence reasoning, it means typed evidence chains, rescue-focused explanation, and path-level scoring rather than treating every problem as a flat classifier.

Both threads point toward the same general methodology: *design the intermediate objects so that the proofs and explanations become natural.* When that is done well, the most difficult work shifts from ad hoc debugging to controlled, reusable theorem and model construction.